Table of Contents

Introduction ---- 2
Getting started ---- 4
Smart UI Scaler editor window ---- 7
Smart UI Scaler component ---- 10
UI Scaler groups ---- 12
Smart UI Expander component ---- 15
Smart UI Size Limiter component ---- 18
Custom UI Scaler ---- 19
Staying in sync with UI Scalers ---- 21
Demo scenes ---- 22
Release notes ---- 24

Introduction

Smart UI Scaler makes one Unity UI (uGUI) layout fit every screen. Design your UI once at a reference resolution, and Smart UI Scaler fits it into any screen size, aspect ratio, safe area or camera viewport, growing it towards the edges exactly as far as you allow. No more choosing between a layout that is cut off on phones and one that floats in the middle of an ultra wide monitor.

Why not the Canvas Scaler?

Unity's Canvas Scaler scales the whole canvas by one rule. Everything stretches or shrinks together, wide screens get empty margins or cropped edges, the safe area is your problem, and a single canvas cannot serve a split screen. Smart UI Scaler works per Rect Transform instead: each scaler is a rectangle with its own reference resolution and its own rules, and you can have as many of them as your screen needs.

What you get

- 📐 **Design once, ship everywhere**
  Build the UI at your reference resolution and keep it readable from a phone in portrait to a 32:9 monitor.

- 🛡️ **Safe area done for you**
  The UI stays inside the notch and home indicator insets, follows rotation, and the Smart UI Expander lets bars and backgrounds still reach the real screen edges.

- 📏 **You decide how the UI expands**
  Expand Progress sets how far the layout grows from the reference resolution to the full screen, Alignment sets where it sits, and the Smart UI Size Limiter keeps a layout between two aspect ratios.

- 🎚️ **A UI scale slider for your players**
  Content Scale shrinks the UI while it keeps covering the same area. Bind it to a settings slider and the whole HUD follows.

- 🧩 **Several scalers on one screen**
  HUD, menus, popups and split screen viewports each scale on their own terms. Scalers can be nested and mixed with regular UI in the same canvas.

- 💾 **Shared settings that persist**
  Group scalers through assets, change their options at runtime and keep them between sessions with the built-in PlayerPrefs persistence or your own save system.

- 🛠️ **See it before you press Play**
  The layout looks the same in Edit Mode and in Play Mode. Preview any aspect ratio, simulate safe areas and switch between device presets from one editor window.

- ⚡ **Nothing to pay per frame**
  Scalers resize only when the screen, the safe area or their options change. In a steady frame the plugin only checks that nothing changed, with no allocations.

- 🔌 **Extensible**
  Write custom scalers for render textures, read scaler sizes in the same frame through an update event, and learn from twelve demo scenes that cover every feature.

What is included

- Smart UI Scaler, Smart UI Expander and Smart UI Size Limiter components
- UI Scaler Groups with runtime API and PlayerPrefs persistence
- Smart UI Scaler editor window with safe area simulation and presets
- Twelve demo scenes, from a single scaler to split screen and custom scalers
- Full source code, online manual and scripting API, offline PDFs

Requirements

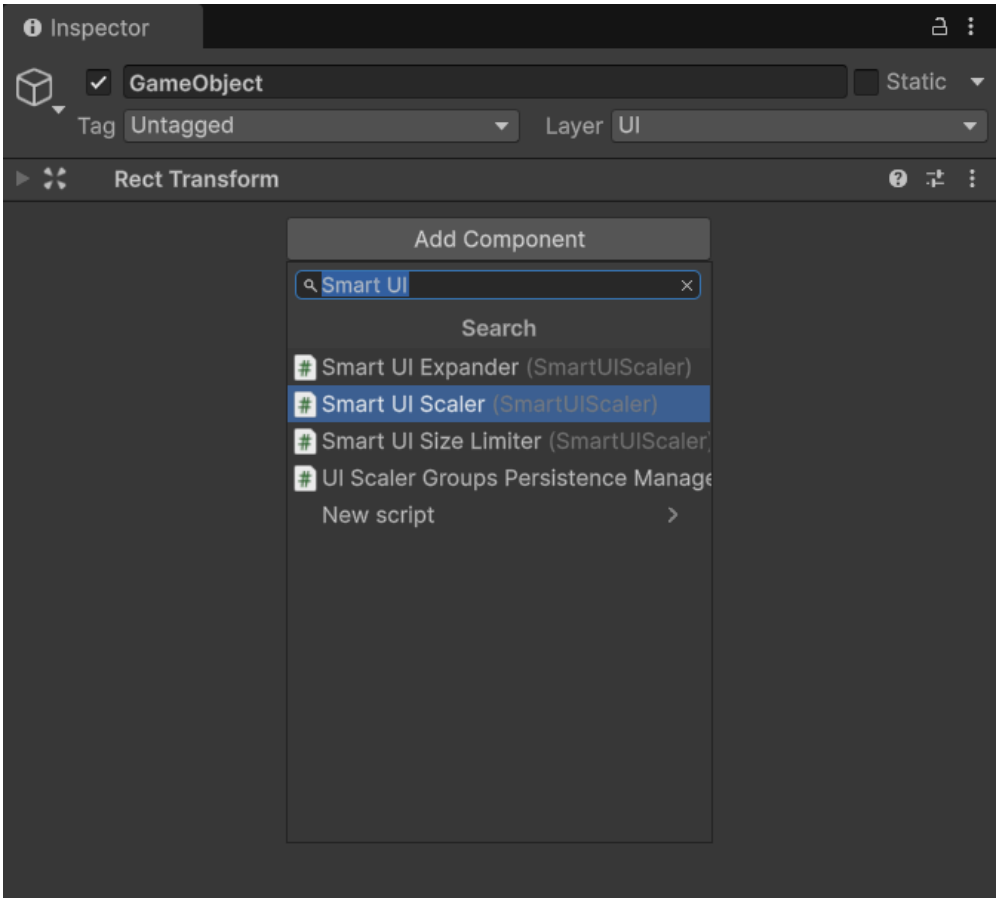
- ❗ **Unity 2022.3.0 or newer**, tested up to Unity 6000.6
- ❗ **Unity UI (uGUI)** canvases, Screen Space Overlay and Screen Space Camera. UI Toolkit is not supported.

Getting started

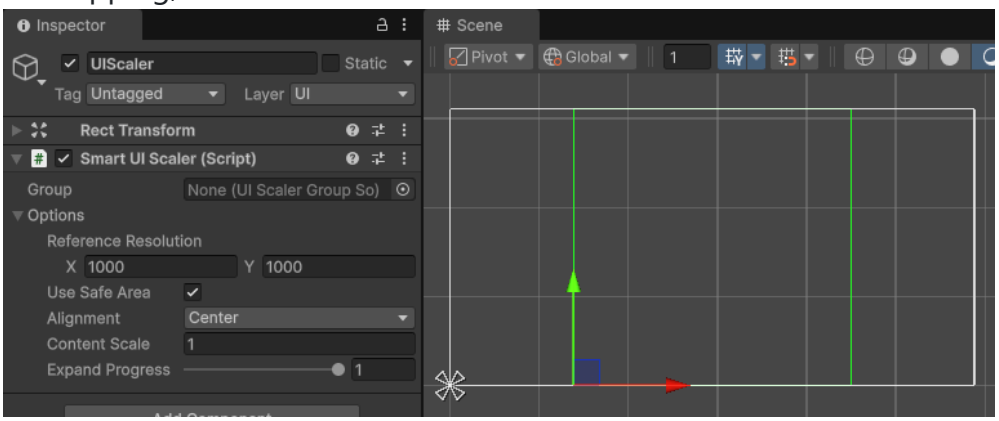
This guide walks you through setting up your first Smart UI Scaler.

1. Create a new GameObject inside your Canvas;

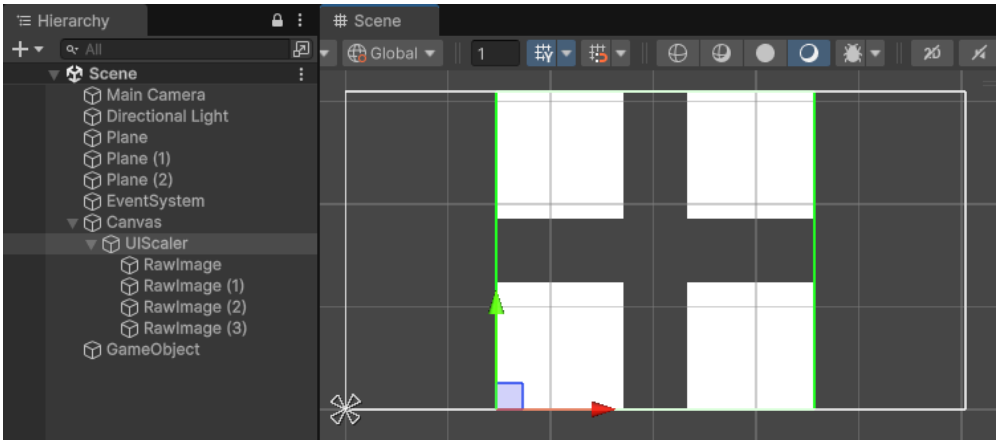
2. Add the Smart UI Scaler component to this GameObject. The Rect Transform of this GameObject will be driven by Smart UI Scaler and will automatically resize;



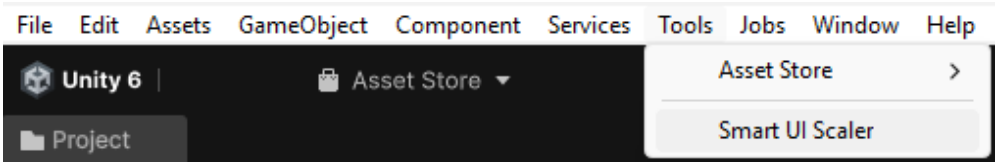
3. At first the Smart UI Scaler will work in "Fitted Reference Resolution" mode. You can see in Scene View that the Canvas is drawn with a gray border and the Smart UI Scaler with a green border. This mode is recommended for editing UI layouts inside Smart UI Scaler and to place objects without overlapping;



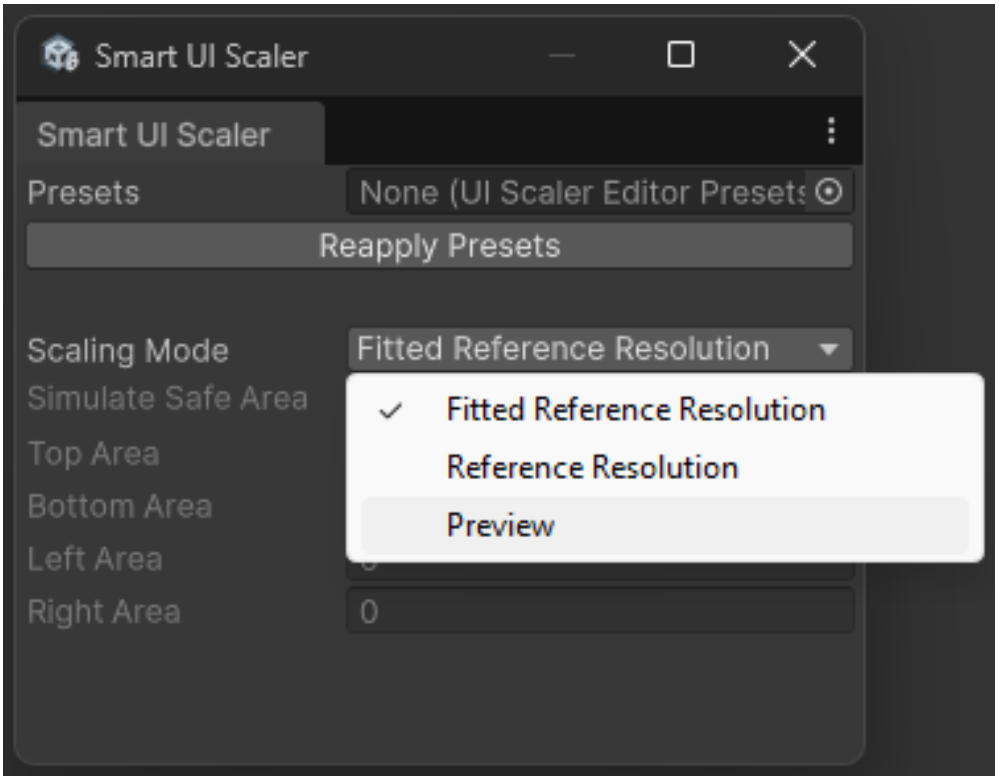
4. Let's add some UI objects to our Smart UI Scaler. As an example, here are four Raw Images anchored to the corners of the Smart UI Scaler;



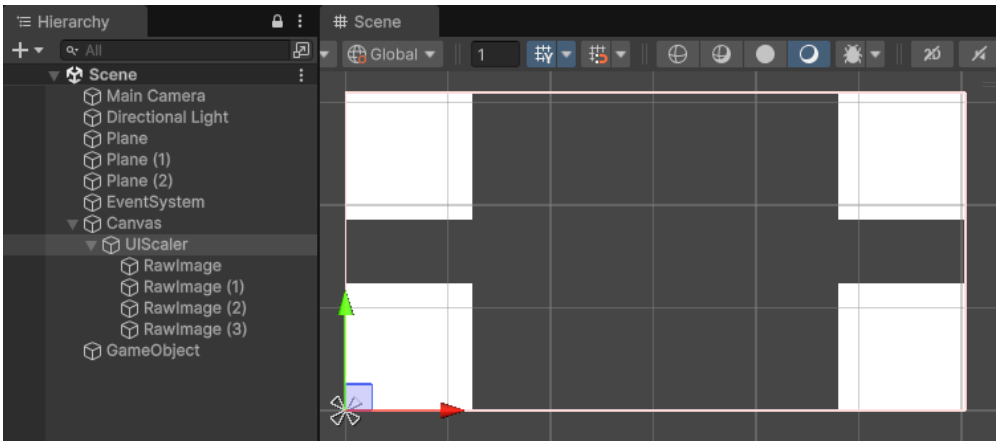
5. Now to test the Smart UI Scaler component with our UI layout open the "Smart UI Scaler" editor window under "Tools" > "Smart UI Scaler";



6. This window helps to test scaling without entering Play Mode. Change the Scaling Mode to "Preview". This mode works exactly as it would in Play Mode;



7. Our Smart UI Scaler now takes up the full area of the Canvas. Now you can test all the Smart UI Scaler component options to see them in effect;



> **NOTE**
>
> In "Preview" mode the Smart UI Scaler border turns red. This is a reminder not to edit the inner UI layout in this mode, because the layout is scaled and its size depends on the Game view.

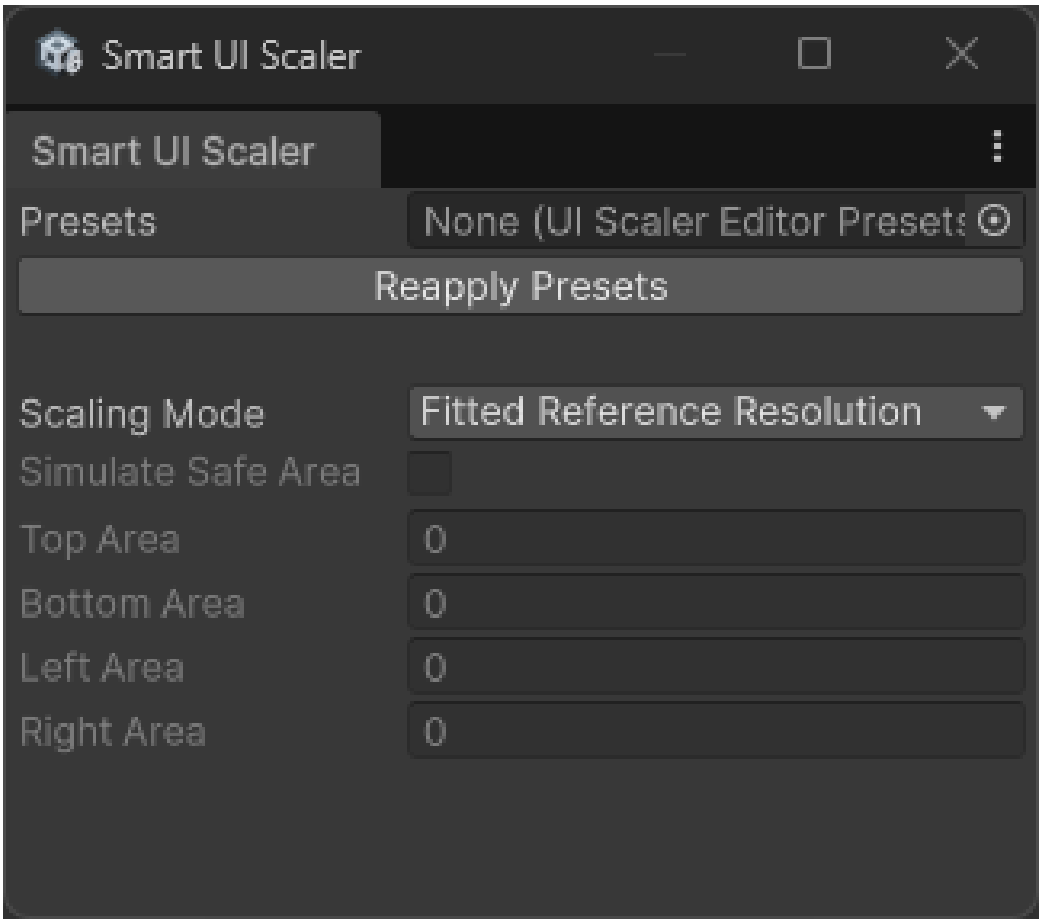
8. Change back the "Scaling Mode" in "Smart UI Scaler" editor window to "Fitted Reference Resolution". The Smart UI Scaler resizes back to the reference resolution and you can continue to work on your UI layout. At this point you can remove the Canvas Scaler component from the Canvas, it is no longer needed.

> **NOTE**
>
> As an alternative, closing the "Smart UI Scaler" window also resets the "Scaling Mode" to "Fitted Reference Resolution".

These steps cover the basics of creating and testing a UI layout. The demo scenes show every option in action.

Smart UI Scaler editor window

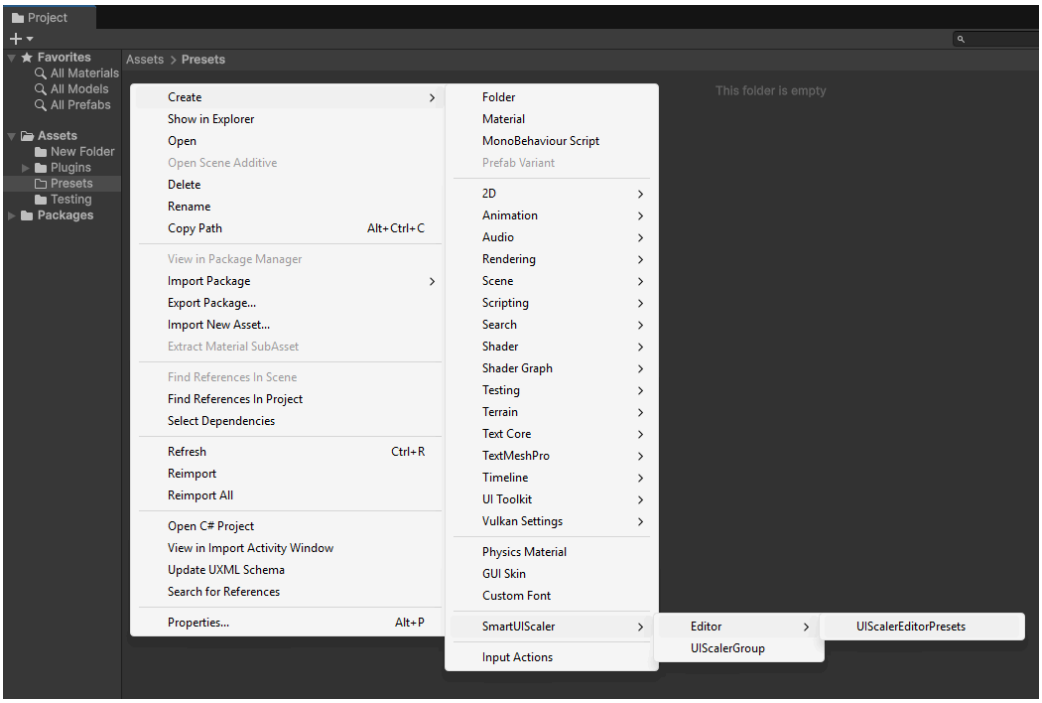
The Smart UI Scaler window helps to test and debug UI layouts without entering Play Mode. It can be found under "Tools" > "Smart UI Scaler";



Presets field

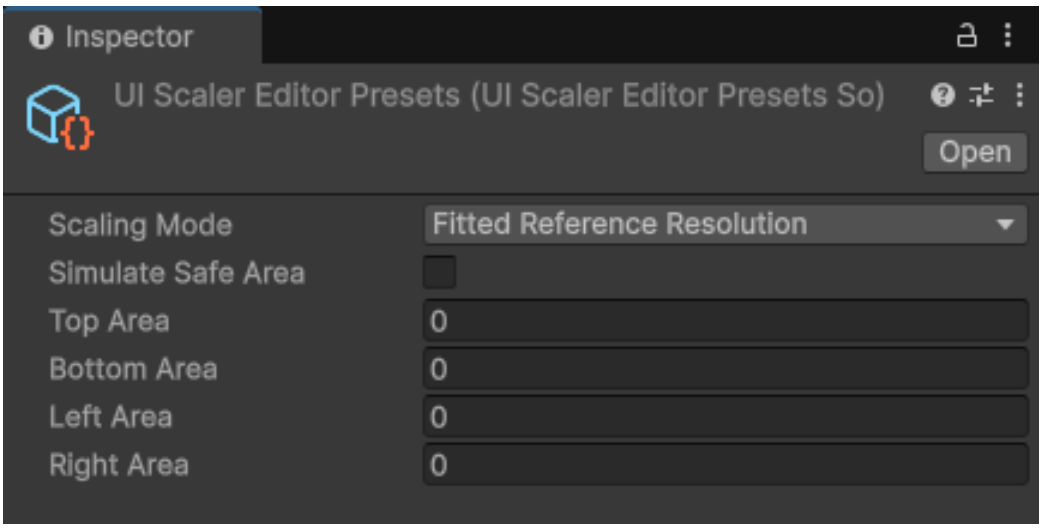
This field is used to apply preset values to Smart UI Scaler window using presets scriptable object.

To create a new presets scriptable object Right Click on Project Window then navigate to "SmartUIScaler" > "Editor" > "UIScalerEditorPresets".



> **WARNING**
>
> UIScalerEditorPresets scriptable object should be placed inside "Editor" folder as this scriptable object only works in Editor.

Presets scriptable object has the same options as Smart UI Scaler window.



Presets options will be applied when Presets scriptable object is selected. They can be reapplied by pressing the "Reapply Presets" button.
The plugin ships with three presets in `Demos/Common/Editor`: "Preview Presets", "iPhone 12 Portrait Presets" and "iPhone 12 Landscape Presets". The iPhone presets enable the safe area simulation with the notch and home indicator paddings of that device.

Scaling Mode field

Scaling Mode field controls how UI Scalers are scaled in Edit Mode.

Scaling Mode	Description
Fitted Reference Resolution	Sets the size of UI Scalers to reference resolution and scales them to fit inside their Canvases. This mode is recommended for UI development and is most convenient to work with.
Reference Resolution	Sets the size of UI Scalers to reference resolution and keeps the scale to represent their reference resolution 1:1 in Unity units. This mode is also intended for UI development.
Preview	Enables full UI Scalers resizing that represents how UI Scalers will look like in Play Mode. It is not recommended to edit the inner UI layout when this mode is enabled.

> **NOTE**
>
> This field only works in Edit Mode and is ignored in Play Mode. The "Fitted Reference Resolution" and "Reference Resolution" modes center the UI Scaler inside its Canvas and ignore the Alignment option, which only applies in "Preview" mode and in Play Mode.

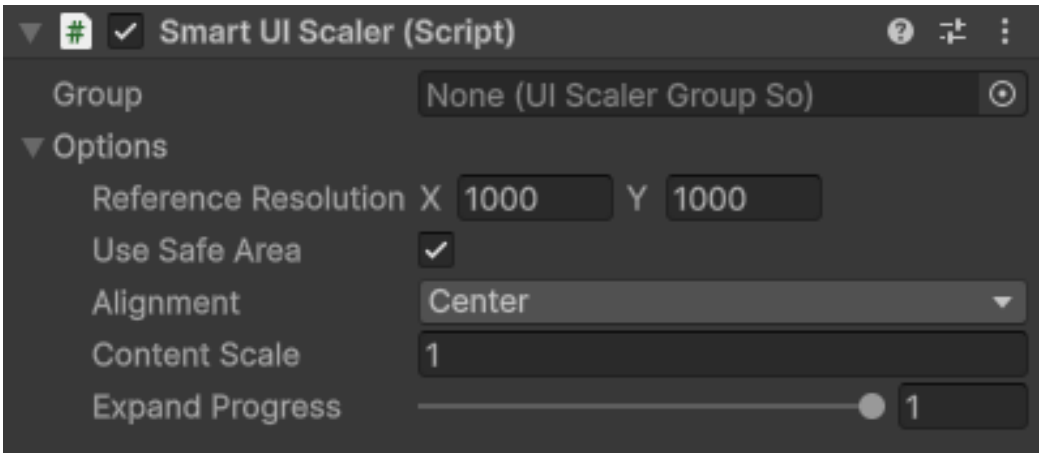
Simulate Safe Area and Area fields

These fields control the simulated screen safe area. To activate the safe area simulation enable the "Simulate Safe Area" toggle. The Area fields control the virtual displays borders padding around the safe area.

> **NOTE**
>
> Safe area simulation fields also work in Play Mode.

Smart UI Scaler component

The main component of this plugin that replaces the built-in Canvas Scaler component.



Group field

Used to assign this Smart UI Scaler into a shared group with shared options. For more information see UI Scaler groups.

Options fields

These options are used when this Smart UI Scaler is not assigned into a group.

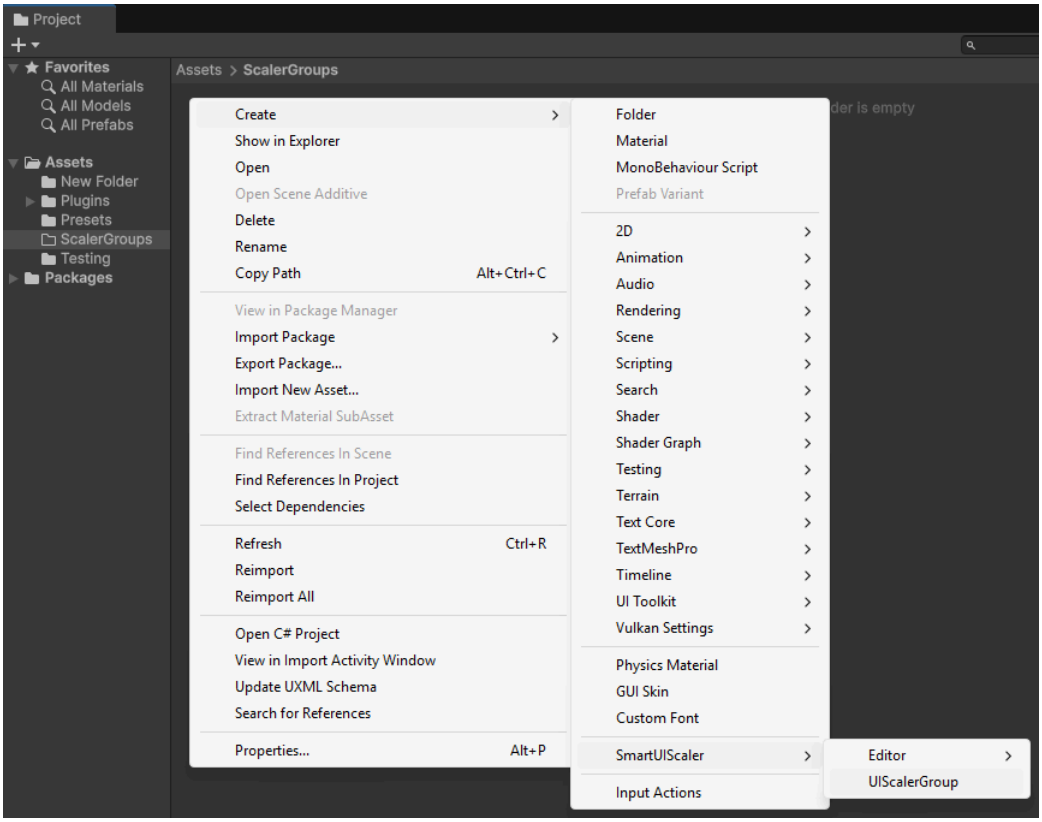
Field	Description
Reference Resolution	The resolution your UI layout is designed for. In Edit Mode the UI Scaler has exactly this size.
Use Safe Area	Fit into the screen's safe area instead of the whole screen area.
Alignment	Where the UI Scaler sits inside the Canvas area when it does not fill it, that is when Expand Progress is below 1.
Content Scale	Additional scale applied to the children. 1 is the designed size, lower values shrink the UI while it keeps covering the same area. Minimum is 0.1, recommended range is 0.1 to 1.
Expand Progress	How far the UI Scaler grows from the reference resolution (0) towards the whole available area (1). Allowed range is 0 to 1.

WARNING

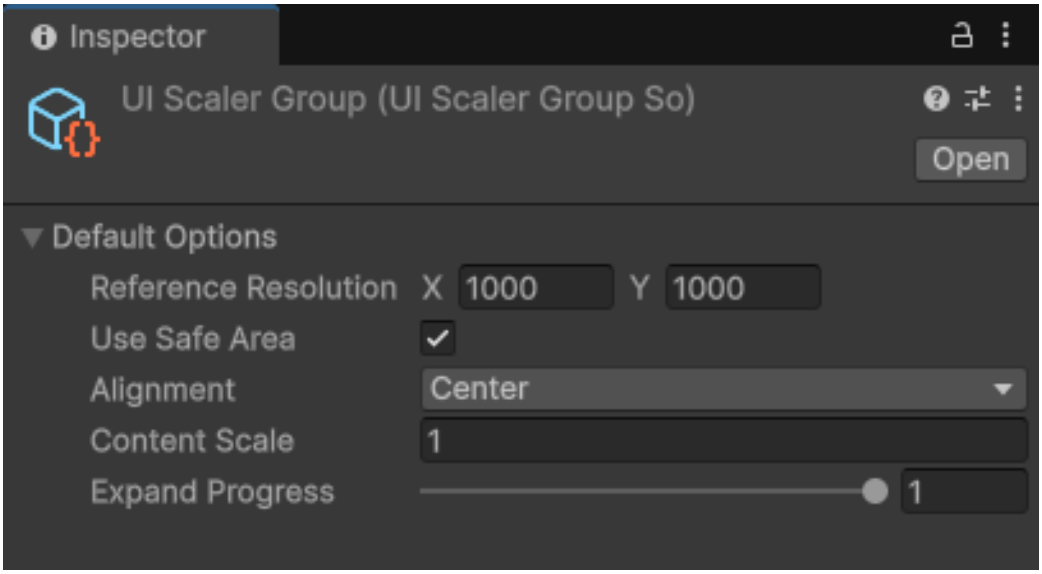
This UI Scaler only works in Canvases with "Overlay Screen Space" Render Mode or "Camera Screen Space" Render Mode where attached camera is rendering directly to the screen. When the attached camera renders to a Render Texture this UI Scaler cannot know the screen space area of the rendered image and this information has to be additionally provided.

UI Scaler groups

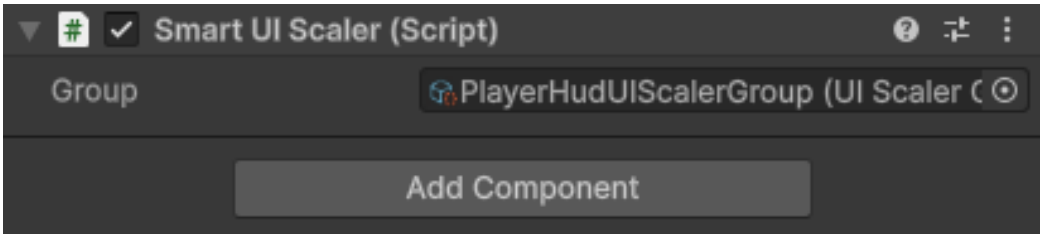
UI Scaler Groups are used to share and control UI Scaler options between UI Scalers. UI Scaler Groups are controlled through their scriptable objects. To create a UI Scaler Group scriptable object Right Click on Project Window then navigate to "SmartUIScaler" > "UIScalerGroup".



The UI Scaler Group scriptable object has a "Default Options" dropdown with the same fields as the Smart UI Scaler component. These are the options every UI Scaler in the group starts with. At runtime they can be changed through the scripting API and the changed values apply to all UI Scalers in the group.



To assign a Smart UI Scaler to this group drag UI Scaler Group scriptable object into Smart UI Scaler "Group" field. The Smart UI Scaler "Options" field disappears, as the group options are used instead.



UI Scaler Group options can be changed through scripting API using the UI Scaler Group scriptable object reference (UIScalerGroupSo API). Once the options are changed - all active UI Scalers will automatically update. Check out the Demo 9 inside the plugin, where a complete UI Scaler Group example is showcased.

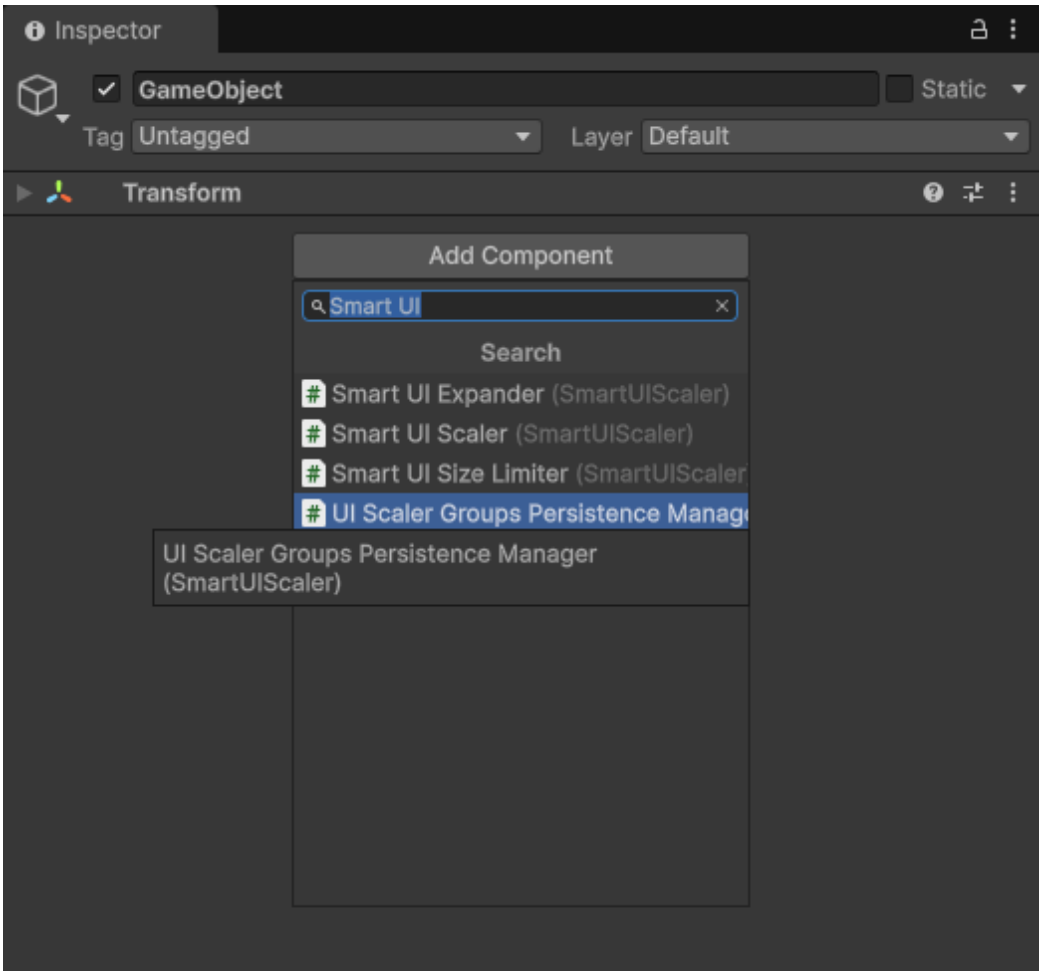
Save Id field

The "Save Id" field identifies the group in saved data. It is generated when the scriptable object is created, so renaming or moving the asset does not affect saved settings. The "New" button generates a fresh Save Id, which detaches the group from any settings saved under the old one.

The editor keeps Save Ids unique across the project and shows a warning when two groups share one, because only one of them would be saved.

Persistence

Initially UI Scaler Group options are not saved and loaded. To have persistent UI Scaler Group options the easiest way is to use UI Scaler Groups Persistence Manager component. Just add this component to an empty GameObject in the first scene in your project. It loads the options from PlayerPrefs when it awakes and saves them when the game is paused or closed. The "Save Key" field is the PlayerPrefs key used for this.

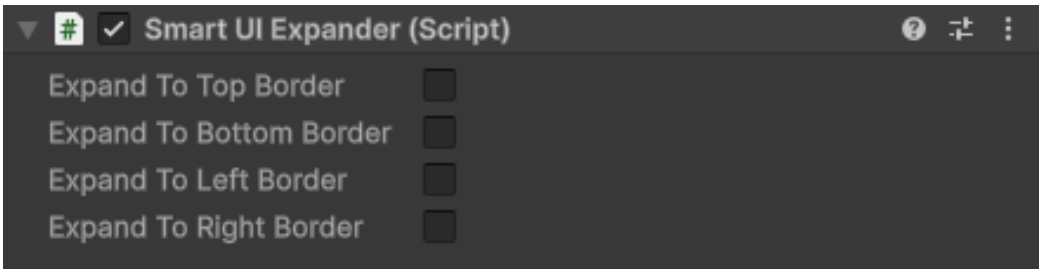


For more advanced scenarios a custom saving and loading can be implemented through UIScalerGroups class GetSaveData() and SetSaveData(UIScalerGroupsSaveData) static methods.

> **WARNING**
>
> Settings saved by version 1.0 are not restored after upgrading to 1.1.0. Version 1.0 identified groups differently, so each group starts from its "Default Options" once and is saved in the new format from then on.

Smart UI Expander component

Used to expand Rect Transform outside of the parent UI Scaler to match parent Canvas borders.



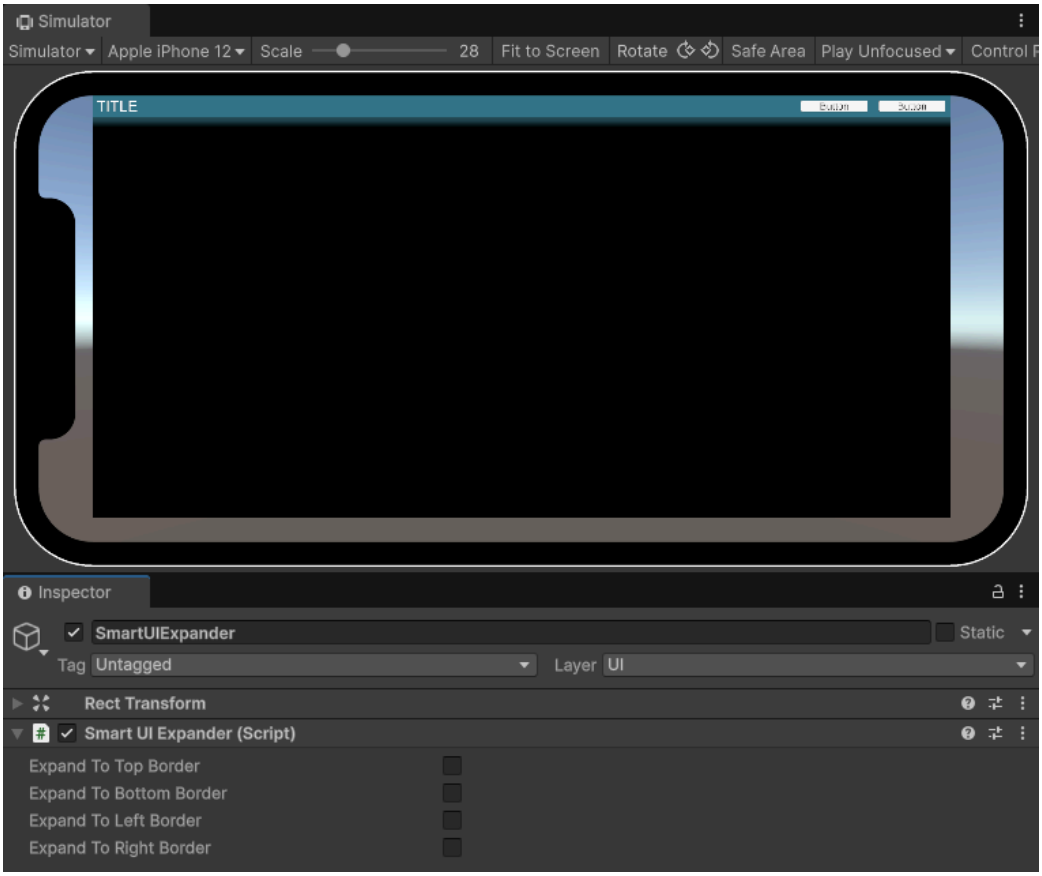
How it works

Smart UI Expander component depends on parent Rect Transform. It follows the parent Rect Transform's size and uses it as the base from which to expand to the Canvas borders. The four expansion fields are used to tell the Smart UI Expander to what directions it should expand.

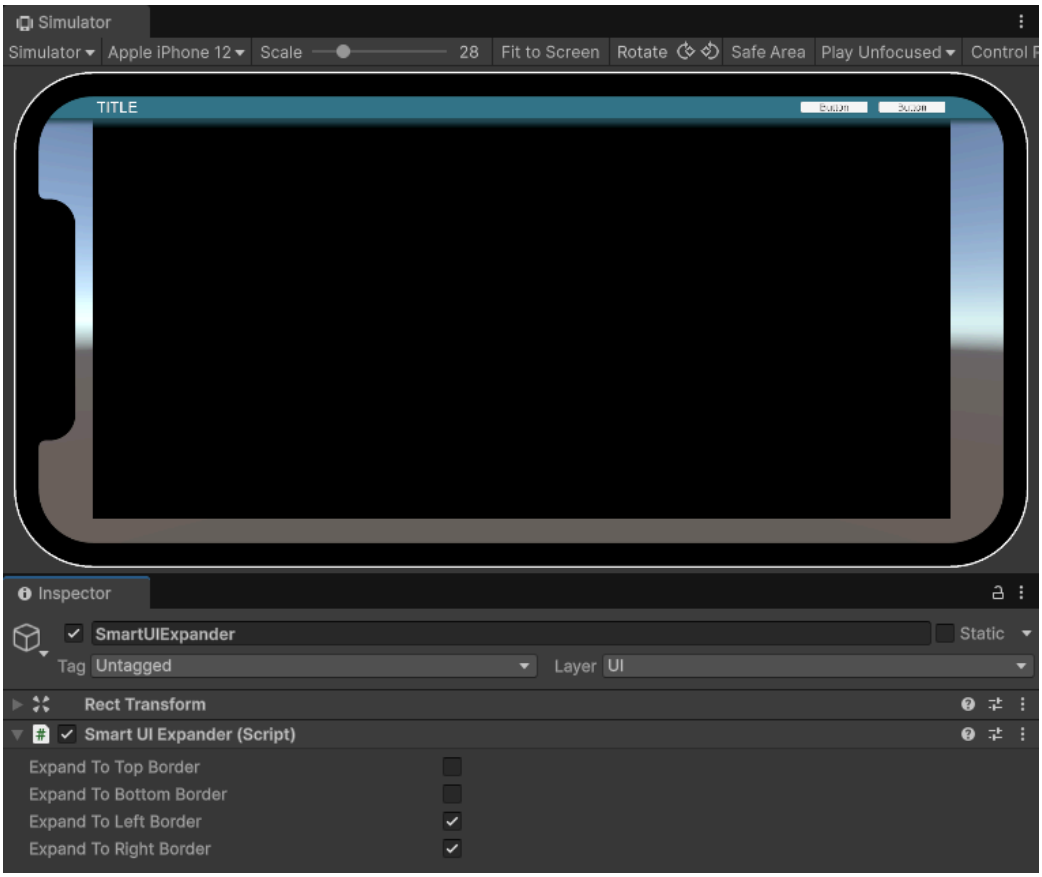
Showcase

Here is a simple example. The black rectangle represents the safe area, it goes up all the way to the top border of the screen. The header is at the top and its background with teal color has the Smart UI Expander component.

In the first image you can see that the header is inside the safe area and the Smart UI Expander does not do anything as no expansion field is selected.



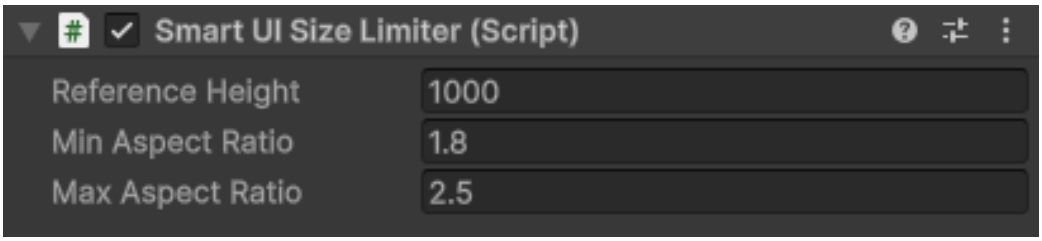
After we set the expansion to left and right borders of the screen we can now see that the header expands outside the safe area to full length of the screen.



You can find the implementation of this example inside the Demo 6 of the plugin.

Smart UI Size Limiter component

Limits the size of the Rect Transform to assigned min and max aspect ratios.



Field	Description
Reference Height	Height set on the Rect Transform and used to calculate the width for the current aspect ratio. Must be greater than 0.
Min Aspect Ratio	Minimum limit for aspect ratio. For unlimited min aspect ratio - set the value to 0.
Max Aspect Ratio	Maximum limit for aspect ratio. For unlimited max aspect ratio - set the value to 0.

How it works

Smart UI Size Limiter component depends on parent Rect Transform. It uses the parent Rect Transform's size to calculate the aspect ratio of its own Rect Transform. If the parent's aspect ratio is between the min and max limits the full parent size is occupied, otherwise the size is limited to the nearest limit.

> **NOTE**
>
> Smart UI Size Limiter component can work without UI Scaler in parent hierarchy.

Demo 7 shows a lobby layout kept between a min and a max aspect ratio.

Custom UI Scaler

A custom UI Scaler is needed when the parent Canvas has "Camera Screen Space" Render Mode and the attached camera renders into a Render Texture. The Smart UI Scaler component cannot know where the rendered image ends up on the screen, so a custom UI Scaler provides that information.

Demo 11 contains a complete implementation, `RenderTextureUIScaler`, for a canvas rendered into a Render Texture and shown by a RawImage. Use it as the reference for the steps below.

Implementation

Extend CoreUIScaler and implement these two methods:

Provide the screen space Rect of the area where the Render Texture is shown.

```
protected abstract Rect GetScreenRect()
```

Convert a screen space Rect to a world space Rect inside the Render Texture's Canvas.

```
protected abstract Rect ScreenRectToWorldRect(Rect screenRect)
```

Mark the class with `[ExecuteAlways]` so that the custom UI Scaler also works in Edit Mode, like the built-in components do.

Detecting changes

A UI Scaler resizes only when its Rect Transform has changed, which is what happens when the Canvas resizes. When the provided screen Rect can change on its own, for example because the RawImage moves on the screen, the custom UI Scaler has to request a resize itself. Override `OnPreWillRenderCanvases`, compare the current screen Rect with the previous one and set `transform.hasChanged = true` before calling the base method:

```
protected override void OnPreWillRenderCanvases()
{
        var screenRect = GetScreenRect();

        if (screenRect != _lastScreenRect)
        {
                _lastScreenRect = screenRect;
                transform.hasChanged = true;
        }
```

```
        base.OnPreWillRenderCanvases();
}
```

Inspector

The Group and Options fields of a UI Scaler are drawn by a custom inspector. To show them on a custom UI Scaler, add a nested editor class that extends `CoreUIScalerEditor` and append the custom fields after the base inspector:

```
#if UNITY_EDITOR
[UnityEditor.CustomEditor(typeof(MyUIScaler))]
class Editor : CoreUIScalerEditor
{
        public override void OnInspectorGUI()
        {
                base.OnInspectorGUI();

                serializedObject.Update();

UnityEditor.EditorGUILayout.PropertyField(serializedObject.FindProperty("_outputImage"));
                serializedObject.ApplyModifiedProperties();
        }
}
#endif
```

Safe area

The "Use Safe Area" option intersects the provided screen Rect with the screen's safe area, so it only makes sense when the screen Rect is in real screen coordinates. Demo 11 uses it this way: the inner UI Scaler only occupies the part of the image that is inside both the screen and the safe area. Turn the option off when the output is not shown directly on the screen.

Staying in sync with UI Scalers

UI Scalers are resized by SmartUIManager just before the canvases are rendered, after all `Update()` and `LateUpdate()` calls of the frame. Code that reads a UI Scaler's size or position in `Update()` therefore sees the values of the previous frame, which shows as a one frame lag whenever the screen, the safe area or the camera viewport changes.

To read UI Scaler sizes in the same frame they were updated, subscribe to AfterScalersUpdatedEvent. It is invoked every frame right after all UI Scalers have been updated, in Play Mode and in Edit Mode.

```
using SmartUIScaler;
using UnityEngine;

public class ScalerFollower : MonoBehaviour
{
        [SerializeField]
        private RectTransform _scaler;

        private void OnEnable()
        {
                SmartUIManager.AfterScalersUpdatedEvent += OnAfterScalersUpdated;
        }

        private void OnDisable()
        {
                SmartUIManager.AfterScalersUpdatedEvent -= OnAfterScalersUpdated;
        }

        private void OnAfterScalersUpdated()
        {
                var size = _scaler.rect.size;
        }
}
```

Demo 12 shows the difference between a follower positioned in this event and one positioned in `Update()`.

> **NOTE**
>
> `SmartUIManager` is created automatically when the game starts and must not be added to a GameObject or destroyed manually.

Demo scenes

The plugin ships with twelve demo scenes under `Assets/Plugins/SmartUIScaler/Demos`, ordered from the most basic concept to the advanced ones. Every scene has an instructions panel that explains what to look at.

Most demos are meant to be explored without entering Play Mode: open "Tools" > "Smart UI Scaler", set the Scaling Mode to "Preview" and change the Game view aspect ratio.

Demo	Scene	What it shows
1	Single Scaler	One Smart UI Scaler with Expand Progress 1 that always fills the safe area. Markers stay anchored to its corners and edges while the aspect ratio changes.
2	Reference Resolution and Safe Area	The same HUD layout in a 1600 x 900 and in a 900 x 1600 Reference Resolution, both with Expand Progress 0, next to Use Safe Area on versus off. Enable "Simulate Safe Area" in the editor window to see the difference.
3	Alignment	Nine Smart UI Scalers, one per Alignment value, each in its own Screen Space Camera canvas. In Play Mode every viewport animates between wide and tall.
4	Expand Progress	Three centered Smart UI Scalers with Expand Progress 0, 0.5 and 1.
5	Content Scale	Two Smart UI Scalers with Content Scale 1 and 0.5. The boxes shrink, the covered screen area stays the same.
6	Smart UI Expander	A header background that uses Smart UI Expander to reach the canvas borders beyond the safe area.
7	Smart UI Size Limiter	A lobby layout kept between a min aspect ratio of 1.8 and a max aspect ratio of 2.5.
8	Nested Behaviours	A Smart UI Expander and a Smart UI Size Limiter nested inside a Smart UI Scaler.
9	UI Scaler Groups	A HUD and a portrait, two Smart UI Scalers bound to the same group. In Play Mode press Escape to open a menu whose sliders change the group's Expand Progress and Content Scale. The settings persist through PlayerPrefs.

Demo	Scene	What it shows
10	Split Screen	Two Screen Space Camera canvases, one per split screen camera, each with its own Smart UI Scaler fitted to its camera viewport.
11	Custom UI Scaler	A canvas rendered into a Render Texture and shown by a RawImage, with a custom UI Scaler inside that only uses the part of the image inside the screen and the safe area. See Custom UI Scaler.
12	After Scalers Updated Event	A Smart UI Scaler in an animated camera viewport, followed by two frames on another canvas: one positioned in `SmartUIManager.AfterScalersUpdatedEvent` stays in sync, one positioned in `Update()` trails a frame behind. See Staying in sync with UI Scalers.

NOTE

The demos use TextMeshPro for their instruction panels. If the TextMeshPro essential resources are not in your project yet, Unity offers to import them when a demo scene is opened.

Release notes

1.1.0

New

- Unity 6 support, tested up to Unity 6000.6. The minimum version stays Unity 2022.3.0.
- `SmartUIManager.AfterScalersUpdatedEvent` is invoked every frame after all UI Scalers are updated. Use it to read UI Scaler sizes in the same frame. See Staying in sync with UI Scalers.
- UI Scaler Group scriptable objects have a "Save Id" field that identifies the group in saved data. It is generated automatically and kept unique by the editor.
- UI Scaler Groups Persistence Manager also saves when the application is paused, so settings survive on mobile when the app is closed from the background.
- Twelve demo scenes, from the basic concepts to split screen and custom scalers. See Demo scenes.

Changes

- UI Scalers refit when the safe area changes without a resolution change, for example when a phone with a camera cutout is turned upside down.
- Content Scale is limited to a minimum of 0.1 and Expand Progress to the range 0 to 1.
- The `Scripts/Internal` folder of the plugin is now `Scripts/Core`. If an empty `Scripts/Internal` folder is left in the project after upgrading, delete it.

> **WARNING**
>
> Settings saved by version 1.0 through the UI Scaler Groups Persistence Manager are not restored by 1.1.0. Each group starts from its "Default Options" once and is saved in the new format from then on.

1.0

Initial release.